Artificial Intelligence

1.Design and Implement a Simple Reflex Agent for the Vacuum Cleaner World and Analyze Its Rational Behavior in a Given Environment

```
# Simple Reflex Agent for Vacuum Cleaner World

A = input("Is Room A dirty? (yes/no): ").lower()
B = input("Is Room B dirty? (yes/no): ").lower()
location = input("Vacuum location (A/B): ").upper()

if location == 'A':
    if A == 'yes':
        print("Suck dirt from Room A")
        A = 'no'

    if B == 'yes':
        print("Move Right to Room B")
        print("Suck dirt from Room B")
        B = 'no'

elif location == 'B':
    if B == 'yes':
        print("Suck dirt from Room B")
        B = 'no'

    if A == 'yes':
        print("Move Left to Room A")
        print("Suck dirt from Room A")
        A = 'no'

if A == 'no' and B == 'no':
    print("Both rooms are clean!")
```

Viva Questions

1. What is an intelligent agent?
An intelligent agent is a system that observes its environment and takes actions to achieve a goal.

2. What is a Simple Reflex Agent?
A Simple Reflex Agent selects actions based only on the current percept using condition-action rules.

3. What is the difference between an agent and an environment?

- **Agent:** The system that takes actions.
- **Environment:** The surroundings in which the agent operates.

4. What are sensors and actuators?

- **Sensors:** Used to observe the environment.
- **Actuators:** Used to perform actions in the environment.

5. What is PEAS?

PEAS stands for **Performance Measure, Environment, Actuators, and Sensors**. It describes an AI agent's task environment.

6. What is the performance measure for a vacuum cleaner agent?

Its performance measure is to keep rooms clean while minimizing time, energy, and unnecessary movements.

7. Why is the vacuum cleaner considered an intelligent agent?

Because it observes whether a room is dirty or clean and takes appropriate actions, such as cleaning or moving.

8. What is meant by rational behavior?

Rational behavior means choosing the best possible action to achieve the goal based on the available information.

9. What are the limitations of a Simple Reflex Agent?

It cannot remember past actions and makes decisions only using the current percept.

10. How is a Model-Based Reflex Agent different from a Simple Reflex Agent?

A Simple Reflex Agent uses only the current percept, while a Model-Based Reflex Agent maintains an internal state to track the environment.

11. Can a Simple Reflex Agent remember previous actions?

No, a Simple Reflex Agent does not maintain memory of previous actions or percepts.

12. What happens if both rooms are clean?

The agent does not need to clean either room. It may stop or take no action, depending on the program's logic.

13. Why is if-elif-else used in this implementation?

It checks conditions and selects the appropriate action based on the current situation.

14. What are the possible percepts in the Vacuum Cleaner World?

The possible percepts are:

- Current location: A or B.
- Current room status: Dirty or Clean.

Quick example: If the agent is in Room A and it is dirty, the agent performs the **Suck** action to clean it.

Practical- 02

2.Implement A* algorithm for 8 puzzle problem

```
from heapq import heappush, heappop

goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def h(s):
    return sum(abs(i//3 - (v-1)//3) + abs(i%3 - (v-1)%3)
               for i, v in enumerate(s) if v != 0)

def astar(start):
    q = [(h(start), 0, start, [start])]
    visited = set()

    while q:
        f, g, s, path = heappop(q)

        if s == goal:
            return path

        if s in visited:
            continue
        visited.add(s)

        z = s.index(0)
        r, c = z//3, z%3

        for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
            nr, nc = r+dr, c+dc

            if 0 <= nr < 3 and 0 <= nc < 3:
                j = nr*3 + nc
                t = list(s)
                t[z], t[j] = t[j], t[z]
                t = tuple(t)

                if t not in visited:
                    heappush(q, (g+1+h(t), g+1, t, path+[t]))

    return None

start = tuple(map(int, input("Enter initial state (9 numbers, 0 for blank): ").split()))

print("\nInitial State:")
for i in range(0, 9, 3):
    print(start[i:i+3])

print("\nGoal State:")
```

```
        for i in range(0, 9, 3):
            print(goal[i:i+3])

        path = astar(start)

        if path:
            for step, state in enumerate(path):
                print("\nStep", step, ":")
                for i in range(0, 9, 3):
                    print(state[i:i+3])


    print("\nTotal Moves:", len(path)-1)
else:
    print("No solution found.")
```

Viva Questions :

1. What are informed and uninformed search techniques?

- **Informed search:** Uses heuristic knowledge to find a solution efficiently.
- **Uninformed search:** Searches without additional knowledge about the goal's direction.

2. List differences between informed and uninformed search techniques.

- **Informed search:** Uses heuristic information.
- **Uninformed search:** Does not use heuristic information.
- **Examples:** Informed – A*, Greedy Best-First Search. Uninformed – BFS, DFS.

3. What is the evaluation function of the A* algorithm?

A* uses the formula **f(n) = g(n) + h(n)**, where:

- **g(n):** Cost from the start node to the current node.
- **h(n):** Estimated cost from the current node to the goal.
- **f(n):** Estimated total cost of the path.

4. What are the applications of the A* algorithm?

A* is used in GPS navigation, robot pathfinding, video games, and route planning.

5. What is a problem-solving agent?

A problem-solving agent identifies a goal, searches for a sequence of actions, and performs those actions to achieve the goal.

6. Is the A* algorithm an informed or uninformed search technique?

A* is an **informed search technique** because it uses a heuristic to estimate the cost of reaching the goal.

7. What is the Best-First Search algorithm?

Best-First Search expands the most promising node first based on an evaluation function.

8. Where are searching techniques applied in AI?

They are used in pathfinding, game playing, robotics, route planning, and problem-solving.

9. What are blind search algorithms?
Blind search algorithms are search techniques that explore states without using heuristic information. Examples include BFS and DFS.

10. What is meant by a heuristic?
A heuristic is an estimate used to guide a search algorithm toward the goal more efficiently.
Example: In route finding, the straight-line distance to the destination can be used as a heuristic.

Practical- 03

3. Tower of Hanoi using State Space Search
Implement a state-space search technique to solve the Tower of Hanoi problem and display the sequence of legal moves from the initial state to the goal state.

```
def hanoi(n, source, auxiliary, destination):
    if n == 1:
        print("Move disk 1 from", source, "to", destination)
        return

    hanoi(n-1, source, destination, auxiliary)
    print("Move disk", n, "from", source, "to", destination)
    hanoi(n-1, auxiliary, source, destination)


n = int(input("Enter number of disks: "))

if n <= 0:
    print("Enter a positive number of disks.")
else:
    print("Initial State: All disks on A")
    print("Goal State: All disks on C")

    print("Minimum number of moves required:", 2**n - 1)

    hanoi(n, 'A', 'B', 'C')

    print("Goal Reached!")
```

Viva Questions :

1. What is meant by a state space in Artificial Intelligence?
A state space is the set of all possible states that can occur while solving a problem.

2. What are the initial state and goal state in the Tower of Hanoi problem?
- **Initial state:** All disks are on the first peg.
- **Goal state:** All disks are moved to the third peg in the same order.

3. Which moves are legal in the Tower of Hanoi?

1. Move only one disk at a time.
2. Move only the top disk of a peg.
3. Never place a larger disk on a smaller disk.

4. Why is cycle checking useful in state-space search?
Cycle checking prevents revisiting previously explored states and avoids unnecessary repeated work.

5. How is the path cost calculated in this problem?
The path cost is the total number of moves made to reach the goal. Each move has a cost of 1.

6. What is the minimum number of moves required for n disks?
The minimum number of moves is $\mathbf{2^n - 1}$, where *n* is the number of disks.
Example: For 3 disks, the minimum is $2^3 - 1 = 7$ moves.

7. Why does Breadth-First Search find an optimal solution when every move has equal cost?
BFS explores states level by level, so it finds the goal using the minimum number of moves first.

8. What is the difference between a state and an operator?

- **State:** Represents the current arrangement of disks.
- **Operator:** Represents an action that changes the state, such as moving a disk.

9. How can parent pointers be used to reconstruct the solution path?
Each state stores its parent state. Starting from the goal and following parent pointers backward reconstructs the solution path.

10. What is the time and space complexity of the recursive Tower of Hanoi solution?

- **Time complexity:** $O(2^n)$
- **Auxiliary space complexity:** O(n), due to the recursion stack.

Practical- 04

Local Beam Search for the Travelling Salesman Problem
Implement local beam search to obtain a short tour for the Travelling Salesman Problem (TSP), in which every city is visited exactly once and the salesman returns to the starting city.

```
# Local Beam Search for TSP

import random

cities = {
    'A': (0, 0),
    'B': (2, 3),
    'C': (5, 2),
    'D': (6, 6),
    'E': (1, 5)
}
```

```
def distance(a, b):
    x1, y1 = cities[a]
    x2, y2 = cities[b]
    return ((x1-x2)**2 + (y1-y2)**2)**0.5

def cost(tour):
    total = 0
    for i in range(len(tour)):
        total += distance(tour[i], tour[(i+1) % len(tour)])
    return total

# Local Beam Search
k = 3
beam = []

for i in range(k):
    tour = list(cities.keys())
    random.shuffle(tour)
    beam.append(tour)

for step in range(100):
    candidates = []

    for tour in beam:
        for i in range(len(tour)):
            for j in range(i+1, len(tour)):
                new_tour = tour[:]
                new_tour[i], new_tour[j] = new_tour[j], new_tour[i]
                candidates.append(new_tour)

    candidates.sort(key=cost)
    beam = candidates[:k]

best = min(beam, key=cost)

print("Best Tour:", " -> ".join(best + [best[0]]))
print("Total Distance:", round(cost(best), 2))
```

Viva Questions :

1. What is the Travelling Salesman Problem?
TSP is a problem in which a salesman must visit every city exactly once and return to the starting city with minimum total distance.

2. Why is TSP called a combinatorial optimization problem?
Because it involves finding the best possible tour from many possible combinations of city arrangements.

3. What is local search?
Local search is a technique that improves a solution by exploring nearby solutions.

4. How does local beam search differ from hill climbing?
Hill climbing explores one current solution at a time, while local beam search explores **k solutions simultaneously**.

5. What is the role of beam width k?
Beam width **k** determines how many candidate solutions are retained at each step.

6. Why should duplicate tours be removed?
Duplicate tours are removed to avoid repeated work and to explore more unique solutions.

7. What is a 2-opt move and why is it useful for TSP?
A 2-opt move removes two edges and reconnects them differently to create a potentially shorter tour.

8. Why is the return distance from the last city to the first city included?
Because the salesman must return to the starting city to complete the tour.

9. What is the difference between the current beam best and the global best tour?

- **Current beam best:** The best tour among the solutions currently retained.
- **Global best tour:** The best tour found throughout the entire search.

10. How can random restarts or different beam widths improve the result?
Random restarts explore different starting solutions, while changing the beam width explores different numbers of candidate tours. Both may help find shorter tours.

Practical- 05

Implement BFS for solving the 8-puzzle problem
Problem statement: Use BFS to solve a planning problem. Examples can be 8 puzzle problem, Robot path planning, Blocks world etc.

```
from collections import deque

goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def bfs(start):
    q = deque([(start, [start])])
    visited = {start}

    while q:
        s, path = q.popleft()

        if s == goal:
            return path

        z = s.index(0)

        for p in [z-3, z+3, z-1, z+1]:
            if 0 <= p < 9 and abs(z//3 - p//3) + abs(z%3 - p%3) == 1:
```

```
                t = list(s)
                t[z], t[p] = t[p], t[z]
                t = tuple(t)

                if t not in visited:
                    visited.add(t)
                    q.append((t, path + [t]))

        return None

start = tuple(map(int, input("Enter 9 numbers (0 for blank): ").split()))

print("\nInitial State:")
print(start[:3])
print(start[3:6])
print(start[6:])

print("\nGoal State:")
print(goal[:3])
print(goal[3:6])
print(goal[6:])

path = bfs(start)

if path:
    for step, s in enumerate(path):
        print("\nStep", step)
        print(s[:3])
        print(s[3:6])
        print(s[6:])

    print("\nTotal Moves:", len(path)-1)
else:
    print("No solution found")
```

Viva Questions :

1. What is Breadth-First Search (BFS)?
BFS is a search algorithm that explores nodes level by level, visiting all nodes at one level before moving to the next.

2. Is BFS an informed or uninformed search technique?
BFS is an **uninformed (blind) search technique** because it does not use heuristic information.

3. What data structure is used by BFS?
BFS uses a **Queue (FIFO)**, which means First In, First Out.

4. What is the difference between BFS and DFS?

- **BFS:** Explores level by level and uses a queue.
- **DFS:** Explores deeply along one path and uses a stack or recursion.

5. How is BFS applied to the 8-puzzle problem?
BFS explores possible board arrangements level by level until it finds the goal arrangement.

6. Why is a visited set required in BFS?
A visited set prevents revisiting the same state and avoids unnecessary repeated exploration.

7. Is BFS complete? Explain.
Yes, BFS is complete when the branching factor is finite. It finds a solution if one exists.

8. When does BFS find the shortest path?
BFS finds the shortest path when every move has equal cost.

9. What are the applications of BFS in AI and planning?
BFS is used in pathfinding, puzzle solving, state-space search, and finding the shortest path in unweighted graphs.

10. What are the main limitations of BFS?
BFS requires a large amount of memory and can be slow when the search space is very large.

Practical- 06

Implement DFS for Water Jug Problem

```
# DFS for Water Jug Problem

def dfs(x, y, target, visited, path):
    if (x, y) in visited:
        return False

    visited.add((x, y))
    path.append((x, y))

    if x == target or y == target:
        return True

    a, b = 4, 3

    states = [
        (a, y), (x, b), (0, y), (x, 0),
        (x - min(x, b-y), y + min(x, b-y)),
        (x + min(y, a-x), y - min(y, a-x))
    ]

    for p, q in states:
        if dfs(p, q, target, visited, path):
            return True
```

```
        path.pop()
        return False

    target = int(input("Enter target amount: "))
    path = []

    if 0 <= target <= 4 and dfs(0, 0, target, set(), path):
        print("Solution Path:")
        for state in path:
            print(state)
        print("Goal Reached!")
    else:
        print("Solution not possible")
```

Viva Questions :

1. What is Depth-First Search (DFS)?
DFS is a search algorithm that explores one path as deeply as possible before backtracking.

2. Is DFS an informed or uninformed search technique?
DFS is an **uninformed (blind) search technique** because it does not use heuristic information.

3. Which data structure is used by DFS?
DFS uses a **Stack (LIFO)** or recursion.

4. What is the difference between DFS and BFS?

- **DFS:** Explores deeply first and uses a stack.
- **BFS:** Explores level by level and uses a queue.

5. How is DFS applied to the Water Jug Problem?
DFS explores possible jug states and actions until it finds a state containing the required amount of water.

6. What is backtracking in DFS?
Backtracking means returning to a previous state when the current path cannot lead to a solution.

7. Does DFS always find the shortest path?
No, DFS does not guarantee the shortest path because it explores one path deeply before trying alternatives.

8. Why is a visited set required in state-space search?
A visited set prevents repeated exploration of the same states and helps avoid infinite loops.

9. What are the applications of DFS in AI?
DFS is used in puzzle solving, state-space search, pathfinding, game trees, and planning problems.

10. What are the limitations of DFS?
DFS may follow a long or unproductive path, does not guarantee the shortest solution, and can use excessive time in large search spaces.

Practical- 07

Implement Backtracking Search to Solve the 8-Queens Constraint Satisfaction Problem
Problem statement: Implement backtracking search to solve the 8-Queens Constraint Satisfaction Problem. The program should place eight queens on an 8 × 8 chessboard such that no two queens attack each other.

```
# 8-Queens using Backtracking

N = 8
board = [-1] * N

def safe(r, c):
    for i in range(r):
        if board[i] == c or abs(board[i]-c) == abs(i-r):
            return False
    return True

def solve(r):
    if r == N:
        return True

    for c in range(N):
        if safe(r, c):
            board[r] = c
            if solve(r+1):
                return True
            board[r] = -1

    return False

solve(0)

for r in range(N):
    for c in range(N):
        print("Q" if board[r] == c else ".", end=" ")
    print()
```

Viva Questions :

1. What is a Constraint Satisfaction Problem (CSP)?
A CSP is a problem where we assign values to variables while satisfying all given constraints.

2. What are the variables and domains in the 8-Queens problem?

- **Variables:** 8 queens, one for each column.
- **Domains:** Possible row positions (1 to 8) for each queen.

3. What are the constraints in the 8-Queens problem?
No two queens should be placed in the same row, column, or diagonal.

4. What is backtracking search?
Backtracking is a search technique that tries possible solutions and goes back when a constraint is violated.

5. Why is backtracking required in the 8-Queens problem?
Backtracking helps find a valid arrangement by removing incorrectly placed queens and trying other positions.

6. How do you check whether a queen can be placed safely?
Check that no other queen is present in the same row, column, or diagonal.

7. What is the diagonal constraint for two queens?
Two queens must not be on the same diagonal.
Formula: |row1 - row2| != |col1 - col2|

8. What is the difference between backtracking and brute-force search?
- **Backtracking:** Rejects invalid arrangements early.
- **Brute force:** Tries all possible arrangements without early rejection.

9. Why is recursion commonly used in backtracking?
Recursion makes it easy to try a position, move to the next step, and return if the position is invalid.

10. What are some applications of backtracking in AI?
Backtracking is used in Sudoku, N-Queens, maze solving, graph coloring, and scheduling problems.

Practical No. 08

Title of the Assignment: Develop a Simple Chatbot Using Predefined Rules and Pattern Matching

```
# Simple Chatbot using Predefined Rules

print("Chatbot: Hello! I am a simple chatbot.")
print("Chatbot: Type 'bye' to exit.")

while True:
    user = input("You: ").lower()

    if "hello" in user or "hi" in user:
        print("Chatbot: Hello! How are you?")

    elif "how are you" in user:
        print("Chatbot: I am fine. How can I help you?")

    elif "your name" in user:
```

```
    print("Chatbot: I am a Simple Chatbot.")

elif "time" in user:
    print("Chatbot: Sorry, I cannot tell the current time.")

elif "bye" in user:
    print("Chatbot: Goodbye! Have a nice day!")
    break

else:
    print("Chatbot: Sorry, I don't understand.")
```